

Scout-Plan-Build

Workflow Guide

Context-Efficient Three-Phase Development Workflow
Using Sub-Agents and Strategic Delegation

Generated: 2025-11-05

Source: IndyDevDan Tactical Agentic Coding

Executive Summary

The Scout-Plan-Build workflow is a three-phase development pattern that dramatically reduces context consumption and accelerates development through strategic delegation to specialized sub-agents.

Metric	Traditional Approach	Scout-Plan-Build	Improvement
Context Usage	50-90%	15-25%	70% savings
Development Time	120 minutes	95 minutes	21% faster
Cost per Task	\$2.00	\$1.00	50% cheaper
File Discovery	Manual search	Parallel agents	4x faster

Key Insight: The R&D; Framework (Reduce and Delegate) - Delegate file search and discovery to cheap sub-agents, preserving your primary agent's context for planning and execution.

1. Architecture Overview

Scout-Plan-Build is a three-phase workflow that separates concerns and optimizes model usage at each stage:

Phase	Description	Model Selection	Duration
SCOUT	4 parallel sub-agents search codebase	Cheap models (free/fast)	5 minutes
PLAN	Primary agent designs implementation	Expensive model (reasoning)	30 minutes
BUILD	Primary agent executes changes	Expensive model (code quality)	60 minutes

Core Philosophy

The workflow follows the "R&D; Framework" principle from IndyDevDan:

- **Reduce:** Minimize context consumption in primary agent
- **Delegate:** Offload discovery to specialized cheap sub-agents
- **Preserve:** Save expensive model capacity for high-value work (planning, building)
- **Parallelize:** Run multiple sub-agents simultaneously for 4x speed

2. Scout Phase: Discovery with Sub-Agents

Purpose

The Scout phase delegates codebase discovery to 4 parallel sub-agents using cheap/free models. This preserves the primary agent's context window (saving 80% of context that would be consumed by manual search).

Sub-Agent Architecture

Sub-Agent	Model	Cost	Search Target	Output
Scout 1	Gemini Flash	Free	Frontend components	relevant_files.md
Scout 2	Gemini Light	Free	Backend API routes	relevant_files.md
Scout 3	Codex	\$0.01/1K	Database schemas	relevant_files.md
Scout 4	Claude Haiku	\$0.25/1M	Business logic	relevant_files.md

Implementation

/scout command implementation:

```
# Launch 4 parallel sub-agents
# Each searches different codebase section
# Output: workspace/scout_results.md with file paths and line numbers

Sub-Agent 1 → Search apps/mobile/ for React Native components
Sub-Agent 2 → Search apps/api/ for tRPC procedures
Sub-Agent 3 → Search packages/database/ for schemas
Sub-Agent 4 → Search packages/training-engine/ for business logic

# Consolidate results into single reference document
# Primary agent reads summary (not entire codebase)
# Result: 80% context window saved
```

Output Format

Each sub-agent outputs structured findings to **relevant_files.md**:

```
## Frontend Components (Scout 1)
- apps/mobile/src/screens/WorkoutScreen.tsx:45-120
→ Voice command handling implementation
- apps/mobile/src/components/VoiceButton.tsx:12-85
→ Speech recognition UI component

## Backend API (Scout 2)
- apps/api/src/routers/workout.ts:78-145
→ Workout logging tRPC procedures

## Database Schemas (Scout 3)
- packages/database/neo4j/schemas/exercises.cypher:1-50
→ Exercise node structure and relationships
```

Critical Advantage: Primary agent reads 500-line summary instead of searching 50,000+ lines of code. Context preserved for actual work.

3. Plan Phase: Strategic Design

Purpose

The Plan phase uses the primary agent (expensive model with strong reasoning) to design the implementation based on scout findings. This is where Claude 4.5 Sonnet's reasoning capabilities are essential.

Primary Agent Workflow

- 1 **Read Scout Results:** Load relevant_files.md summary (500 lines vs 50K)
- 2 **Scrape Documentation:** Pull necessary API docs, patterns, examples locally
- 3 **Design Architecture:** Create implementation plan with file structure
- 4 **Generate Specifications:** Detailed requirements for build phase
- 5 **Save to AI_docs/:** Persist plan for reuse and reference

Planning Mode Best Practices

Always use Planning Mode (Shift+Tab) during this phase to:

- Review plans before execution (prevent manifestation hell)
- Catch architectural issues early (2 minutes vs 30 minutes of wasted work)
- Adjust approach based on constraints discovered
- Save tokens by not building wrong thing

Example Plan Output

```
# Voice Workout Feature Implementation Plan

## Architecture
1. Frontend: VoiceWorkoutScreen.tsx
  - React Native Voice integration
  - Real-time transcription display
  - Error handling for speech recognition

2. Backend: workout.router.ts
  - trRPC procedure: logWorkoutViaVoice
  - Input validation with Zod
  - Neo4j integration for exercise lookup

3. State Management
  - React Query for server state
  - Zustand for voice recognition state

## Dependencies
- react-native-voice: ^3.2.4
- @trpc/server: ^11.0.0
- zod: ^3.22.4

## Testing Strategy
- Unit tests: Voice parsing logic
- Integration tests: trRPC procedures
```

- E2E tests: Full voice workflow with Detox

Time Investment: 30 minutes for comprehensive plan saves 2-3 hours of rework from poor architecture.

4. Build Phase: Execution

Purpose

The Build phase executes the plan created in phase 2, using the primary agent's code generation capabilities while maintaining quality and consistency.

Execution Strategy

- 1 **Load Plan:** Reference AI_docs/ specifications
- 2 **Implement by Module:** Break into logical chunks (frontend → backend → tests)
- 3 **Validate Incrementally:** Test each module before moving to next
- 4 **Use Git Worktrees:** Isolate development from main branch
- 5 **Document as You Go:** Update README, inline comments, API docs

Quality Checks

- TypeScript compilation passes (no any types)
- Tests written and passing (>80% coverage target)
- Linting passes (ESLint + Prettier)
- Performance acceptable (60fps animations, <200ms API responses)
- Error handling implemented (user-friendly messages)
- Documentation complete (JSDoc for public APIs)

Git Worktrees Integration

The Build phase benefits from git worktrees for parallel isolated development:

```
# Create isolated worktree for feature
git worktree add ../voice-feature feature/voice-workout

# Develop in isolation
cd ../voice-feature
# Build, test, verify

# Merge when ready
git checkout main
git merge feature/voice-workout

# Clean up
git worktree remove ../voice-feature
```

Benefit: Main branch stays clean while feature is developed. No conflicts, no interruptions.

5. R&D; Framework: Reduce and Delegate

The R&D; Framework is the philosophical foundation of Scout-Plan-Build, addressing the core problem: context window consumption kills development velocity.

The Problem: Context Window Crisis

- System prompts consume 5-10% of context window
- MCP tools (even unused) consume ~2% per tool
- Conversation history consumes 50-90% on complex tasks
- **Autocompact buffer consumes 22% (critical to disable!)**
- File search operations consume massive context

The Solution: Strategic Delegation

Instead of having the primary expensive agent search codebases:

Approach	Context Consumed	Cost	Time	Quality
Traditional (Primary agent searches)	50-90%	\$2.00	120 min	Variable
R&D Framework (Sub-agents search)	15-25%	\$1.00	95 min	Consistent

Critical Configuration

ALWAYS Disable Autocompact (IndyDevDan - Critical Insight):

```
/confia autocompact false
```

This saves 22% of context window (44K tokens on Claude 4.5 Sonnet's 200K limit). Autocompact compresses conversation history automatically, but consumes precious context doing so. For Scout-Plan-Build workflows where every token counts, disable it.

6. Real-World Example: Voice Workout Feature

This example demonstrates the Scout-Plan-Build workflow for implementing a voice-driven workout logging feature.

User Story

As a hybrid athlete, I want to log workout data using voice commands during training so I don't need to interrupt my session to type on my phone.

Phase 1: Scout (5 minutes)

Launch 4 parallel sub-agents to discover relevant files:

```
Sub-Agent 1 (Gemini Flash - Free):
→ apps/mobile/src/screens/WorkoutScreen.tsx
→ apps/mobile/src/components/VoiceButton.tsx
→ apps/mobile/src/hooks/useVoiceRecognition.ts

Sub-Agent 2 (Gemini Light - Free):
→ apps/api/src/routers/workout.ts
→ apps/api/src/services/workoutService.ts
→ apps/api/src/middleware/auth.ts

Sub-Agent 3 (Codex - $0.01):
→ packages/database/neo4j/exercises.cypher
→ packages/database/neo4j/workouts.cypher

Sub-Agent 4 (Haiku - $0.05):
→ packages/training-engine/src/logWorkout.ts
→ packages/training-engine/src/parseVoiceInput.ts

Output: workspace/scout_results.md (550 lines)
Cost: $0.06 total
Time: 5 minutes (parallel execution)
```

Phase 2: Plan (30 minutes)

Primary agent (Claude 4.5 Sonnet) reads scout results and creates implementation plan:

```
Planning Mode Enabled (Shift+Tab)

1. Architecture Review
✓ Existing voice hooks can be reused
✓ Need new tRPC procedure for voice input
✓ Neo4j exercise lookup already exists

2. Implementation Plan
→ Frontend: Extend VoiceButton component
→ Backend: New logWorkoutViaVoice procedure
→ Parsing: Create parseVoiceWorkoutInput function
→ State: Add voice recording state to Zustand

3. Dependencies (none new required)
✓ react-native-voice already installed
✓ tRPC router already configured

4. Testing Strategy
→ Unit: Voice parsing logic
→ Integration: tRPC procedure
→ E2E: Full voice workflow

Plan saved to: AI_docs/voice-workout-feature.md
Cost: $0.40
Time: 30 minutes
```

Phase 3: Build (60 minutes)

Primary agent executes the plan:

```
1. Create git worktree for isolation
git worktree add ../voice-feature feature/voice-workout
```

```
2. Implement frontend (20 minutes)
✓ VoiceButton.tsx - Add workout mode
✓ WorkoutScreen.tsx - Integrate voice button
✓ useVoiceRecognition.ts - Add workout parsing
```

```
3. Implement backend (20 minutes)
✓ workout.router.ts - New tRPC procedure
✓ workoutService.ts - Voice logging logic
✓ parseVoiceInput.ts - Natural language parsing
```

```
4. Write tests (15 minutes)
✓ parseVoiceInput.test.ts - 8 test cases
✓ workout.test.ts - Integration tests
✓ WorkoutScreen.test.tsx - Component tests
```

```
5. Validation (5 minutes)
✓ TypeScript compilation passes
✓ All tests passing (100% coverage)
✓ Linting passes
✓ Manual testing on iOS/Android
```

Cost: \$0.54
Time: 60 minutes

TOTAL COST: \$1.00
TOTAL TIME: 95 minutes

vs Traditional (single agent): \$2.00, 120 minutes

Result: 50% cost savings, 21% time savings, better code quality through structured workflow.

7. Best Practices and Patterns

Model Selection Strategy

Phase	Task	Recommended Model	Rationale
Scout	File discovery	Gemini Flash/Light	Free, fast, good at search
Scout	Code analysis	Codex	Cheap, understands code structure
Scout	Complex patterns	Claude Haiku	Better reasoning than Gemini
Plan	Architecture design	Claude 4.5 Sonnet	Best reasoning capabilities
Build	Code generation	Claude 4.5 Sonnet	Highest quality output
Build	Simple edits	Switch to Cursor	Save expensive tokens

When to Use Scout-Plan-Build

- **Large codebases:** When file discovery would consume 50%+ context
- **Complex features:** Multi-component implementations across frontend/backend/database
- **Unfamiliar codebase:** Learning phase where exploration is needed
- **Refactoring:** Need to understand impact across multiple files
- **Performance optimization:** Requires analyzing patterns across codebase

When NOT to Use Scout-Plan-Build

- **Simple edits:** Single file changes (use Claude Code directly or Cursor)
- **Bug fixes:** When exact file is known (no discovery needed)
- **Documentation:** Writing docs doesn't need codebase search
- **Small projects:** <1000 lines of code (overhead not worth it)
- **Rapid iteration:** Early prototyping where architecture changes frequently

Context Management Tips

- 1 Disable autocompact buffer: `/config autocompact false` (saves 22%)
- 2 Clear context between phases: `clear` or `/compact`

- 3 Monitor usage: **/context** to check current consumption
- 4 Use Planning Mode: **Shift+Tab** to review before execution
- 5 Save work frequently: Preserve plans to AI_docs/ for reuse
- 6 Switch tools strategically: Use Cursor for high-volume simple edits

8. Troubleshooting and Common Pitfalls

Problem: Sub-Agents Find Irrelevant Files

Symptom: Scout phase returns files unrelated to the feature.

Solution: Improve search prompts with specific keywords and file patterns.

```
# Bad prompt
"Find files related to workouts"

# Good prompt
"Find files containing:
- Voice recognition hooks (useVoice, react-native-voice)
- Workout logging functions (logWorkout, saveWorkout)
- Exercise database queries (Neo4j Cypher, exercise nodes)
- tRPC workout procedures (workout.router.ts)"
```

Problem: Plan Phase Takes Too Long

Symptom: 30-minute plan phase extends to 60+ minutes.

Solution: Break down into smaller features, use iterative planning.

Instead of planning entire voice workout system, plan MVP first:

1. Voice recording only (10 min plan)
2. Then add exercise parsing (10 min plan)
3. Then add Neo4j integration (10 min plan)

Problem: Context Window Still Fills Up

Symptom: Even with Scout-Plan-Build, hitting context limits.

Checklist:

- ✓ Autocompact disabled? `/config autocompact false`
- ✓ Context cleared between phases? `c\lear` or `/compact`
- ✓ Scout results summarized? Should be <500 lines, not full files
- ✓ Using free models for scouts? Gemini Flash/Light cost nothing
- ✓ Planning Mode enabled? Prevents token waste on wrong implementations
- ✓ Switching to Cursor for simple edits? Save expensive tokens

Problem: Sub-Agents Don't Execute in Parallel

Symptom: Scouts run sequentially, taking 20 minutes instead of 5.

Solution: Use proper sub-agent invocation (Claude Code feature). Ensure you're using Task tool with `subagent_type` parameter, not sequential commands.

9. Conclusion and Next Steps

Scout-Plan-Build transforms development velocity through strategic context management and model selection. By delegating discovery to cheap sub-agents and preserving expensive model capacity for high-value work, you achieve 50% cost savings and 21% time savings while improving code quality.

Key Takeaways

- **R&D; Framework is fundamental:** Reduce context needs, Delegate to sub-agents
- **Model selection matters:** Free/cheap for discovery, expensive for reasoning
- **Context is precious:** Disable autocompact, clear frequently, use Planning Mode
- **Parallel execution is powerful:** 4 sub-agents = 4x speed for discovery
- **Planning prevents rework:** 30-minute plan saves 2-3 hours of bad implementations
- **Tools complement each other:** Claude Code for planning, Cursor for building

Implementation Checklist

- ■ Configure Claude Code: `/config autocompact false`
- ■ Create custom slash commands: `/scout`, `/plan`, `/build`
- ■ Set up model selection for sub-agents (Gemini Flash/Light, Codex, Haiku)
- ■ Create workspace/scout_results.md template
- ■ Create AI_docs/ directory for plans
- ■ Practice on small feature first (e.g., add button to UI)
- ■ Scale to complex features (e.g., voice workout logging)
- ■ Measure results: Track cost, time, context usage
- ■ Iterate and optimize: Refine prompts, adjust model selection

Additional Resources

- **IndyDevDan** - Tactical Agentic Coding course (source of R&D; Framework)
- **Carl Vellotti** - Hybrid strategy and cost optimization techniques
- **Claude Code Documentation** - Official sub-agent and planning mode guides
- **Context Engineering** - `.claude/` folder structure best practices

Start with one feature. Master the workflow. Scale your development velocity.